Machine Learning for Mechanical Engineers at CoEP Session 2: Python Overview

Session 2: Python

Python Overview

Setup

Setup

Installing Python

- Pre-installed on most Unix, Linux and MAC OS X
- Windows: Python.org or Anaconda (preferred)
- Anaconda has necessary libraries
- Python 3 only: Python 2 reached end-of-life in 2020
- 'pip' and 'conda'

Installing Python via Anaconda Distribution

- Download the latest **Python 3** Anaconda from https://www.anaconda.com/download
- Choose the 64-bit installer for your OS (Windows / macOS / Linux)
- Includes `conda` package manager and all essential scientific libraries
- After install, update: `conda update conda && conda update --all`
- Lightweight alternative: **Miniconda**: installs only Python + conda

Integrated Dev Envt

- **VS Code** https://code.visualstudio.com/: free, lightweight, excellent Python extension
- **PyCharm** https://www.jetbrains.com/pycharm/: full-featured, Community edition is free
- **Spyder** https://www.spyder-ide.org/: MATLAB-like, bundled with Anaconda
- **IDLE**: minimal IDE that comes with python.org distribution

Virtual Environments

- Keeps each project's packages isolated, so installing one project's dependencies doesn't break another's
- Create one with `conda` or plain Python's `venv`
- Rule of thumb: one environment per project

```
# conda
conda create -n myproject python=3
conda activate myproject

# venv (plain Python)
python -m venv myproject
myproject\Scripts\activate     # Windows
source myproject/bin/activate  # Mac/Linux
```

Language Basics & Syntax

Language Basics & Syntax

What is Python?

- Python is an interpreted, object-oriented, high-level programming language with dynamic semantics.
- Python is open source, free and cross-platform.
- Python provides high-level built in data structures.
- Python is useful for rapid application development.
- Python can be used as a scripting or glue language.
- Python emphasizes readability.
- Python supports modules and packages.

Compiled Languages

- Needs entire program
- Translates directly to machine codes
- Exe native and fast
- Usually statically typed
- Types known during compilation
- Change in type : recompilation
- Ideal for compute-heavy tasks
- E.g. C, C++, FORTRAN

Interpreted Languages

- Interpreted on the fly
- No need to compile: can execute right away
- Usually dynamically typed
- Non-syntax errors are detected only in run-time
- Slower than compiled languages
- Ideal for small tasks
- E.g. Python, Perl, PHP, Bash

Note: Python, at the beginning, loosely checks the program. Only at run time, line-by-line, it checks for errors. So, if the error statements are not in the running path, their error does not get reported. So, its a bit relaxed. Thats the objection for its use in production code, where strong type checking and error checking, upfront is essential.

JIT-Compiled Languages

- Between compiled and interpreted
- Code is initially interpreted, hotspots compiled
- Deduce types during compilation, can change in run-time
- Slower than compiled
- E.g. Java, C#

Important features

- Built-in high level data types: `strings`, `lists`, `dictionaries`, etc.
- Usual control structures: `if`, `if-else`, `if-elif-else`, `while`, `for`
- Levels of organization: `functions`, `classes`, `modules`, `packages`
- Extensions in C and C++ possible

The Python shell, I

- Python can be run from "shell", IDE, Notebook
- Shell/Command Line – start writing commands/expressions at the >>> prompt

```
> python

Python 3.5.3 | packaged by conda-forge | (default, May 12
     2017, 16:16:49) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more
     information.
>>>
```

The Python shell, II

- Expressions are evaluated and the result is printed
- Line continuation with a trailing backslash: the prompt changes to '...' on continuation lines and for loops, function definitions, etc.

```
>>> 2+2
4

>>> "hello" + \
... " world!"
'hello world!'
```

Overall Syntax

- Multiple statements on the same line are separated with ";"
- No semicolon at the end of lines.
- Scope is obtained through indentation.
- Always indent next line if ":" is at the end of current line.
- One script is can be run or imported by other modules.

Lines

- Statement separator : a semi-colon,
- Only needed for multiple statements on a line.
- Else, nothing
- Continuation: a back-slash
- But, not necessary in "context" : ( ),{ },[ ]

Blocks and indentation

- Blocks by indentation
- No begin/end brackets
- Indentation is 4 spaces and no hard tabs
- Reduces clutter
- `if`, `while`, `function`, `except`, etc have : at the end
- : must follow with an indent on next line.

Quick Check: Python Language Basics

Think About It

C and Java use braces { } to mark blocks, and indentation is just a human-readable convention on top of that. Python uses indentation itself as the block boundary. What's the tradeoff?

Quick Check: Python Language Basics

Answer

It forces what the code *looks* like and what it *does* to always agree: you can't have a block that visually appears nested one way but executes another, a class of bug that misleading brace placement can cause in C or Java. The tradeoff is that whitespace becomes meaningful: mixing tabs and spaces, or one stray space, can silently change what the code does, without an obvious visual cue like a misplaced brace.

Comments

- Everything after "#" is ignored: no block comments in Python
- Useless/redundant vs. useful comments, and disabling code temporarily

```
v = 5 # assign 5 to v              (redundant, no info)
v = 5 # velocity in meters/second (useful, explains intent
      )

# print("This won't run.")  -- comment out to disable code
print("This will run.")
```

Data Types

Data Types

Assignment

- Assignment creates references, not values: `tmp = "hello "; tmp = 10` the first string will be deallocated
- As in C programming: `x += 1` is valid
- Pre/post increment/decrements: `x++; ++x; x--;--x` are invalid
- Multiple assignment (references to a unique object):`x=y=z=1`
- Multiple assignments: `(x,y,z)=(3.5,5.5,`string`)`
- Example of swapping variables value: `(x,y)=(y,x)`

Intuition

Think of a variable as a name-tag stuck onto an object, not a box holding a copy of it. `tmp = "hello"` sticks the tag `tmp` on the string; `tmp = 10` just moves the tag onto a different object. If two names are stuck on the same object (`x=y=z=1`), changing what one name points to doesn't touch the object the other still points to.

Built-in object types

- Numbers : `3.1415, 1234, 999999999999, 3+4j`
- Strings : `` `spam', ``guido's'' ``
- Lists : `[1, [2, `three'], 4]`
- Dictionaries :`{`food':`spam', `taste':`yum'}`
- Tuples : `(1,`spam', 4, `U')`
- Sets: `{1,2,3,'foo','bar'}`

Numbers

- Integers : `1234, -24, 0`
- Unlimited precision, no separate 'long' type in Python 3: `999999999999`
- Float : `3.1415, 2.7122`
- Oct and hex :`0177, 0x9ff`
- Complex : `3+4j, 3.0+4.0j, 3J`

Strings (immutable sequences)

- single quote `s1 = `egg'`
- double quotes `s2 = ``spam's''`
- triple quotes `block = ```...'''`
- concatenate `s1 + s2`
- repeat `s2 * 3`
- index,slice `s2[i], s2[i:j]`
- length `len(s2)`
- formatting `` ``a {} parrot''.format(`dead') ``
- iteration `for x in s2 # x loop through each character of s2`
- membership `` `m' in s2 ``

f-Strings: Modern String Formatting (Python 3.6+)

Embed variables and expressions directly in strings by prefixing with `f`. f-strings are **faster**, **more readable**, and the **recommended** style for all new Python 3 code.

```
name = "Alice"
age  = 30

# Old %-style (avoid in new code)
print("Hello, %s. You are %d." % (name, age))
# .format() style
print("Hello, {}. You are {}.".format(name, age))
# f-string (preferred since Python 3.6)
print(f"Hello, {name}. You are {age}.")
print(f"Next year: {age + 1}")        # expressions work
      too
print(f"Pi: {3.14159:.4f}")           # format specs work
      too
```

Lists

- Ordered collections of arbitrary objects
- Accessed by offset
- Variable length, heterogeneous, arbitrarily nest-able
- Mutable sequence
- Arrays of object references

```
L = [1, 2, 3, "four"]
L.append(5)
print(L)        # [1, 2, 3, 'four', 5]
print(L[0])     # 1
```

Lists operations

- empty list `L = []`
- create list `L1 = range(4)`
- four items `L2 = [0, 1, 2, 3]`
- nested `L3 = ['abc', ['def', 'ghi']]`
- index `L2[i], L3[i][j]`
- slice `L2[i:j]`, length `len(L2)`
- concatenate `L1 + L2`, repeat `L2 * 3`
- iteration `for x in L2`, membership `3 in L2`
- methods `L2.append(4), L2.sort(), L2.index(1), L2.reverse()`
- shrinking `del L2[k], L2[i:j] = []`
- assignment `L2[i] = 1, L2[i:j] = [4,5,6]`

Dictionaries

- Accessed by key, not offset
- Unordered collections of arbitrary objects
- Variable length, heterogeneous, arbitrarily nest-able
- Of the category mutable mapping
- Tables of object references (hash tables)

```
d = {'name': 'Alice', 'age': 30}
d['city'] = 'Pune'
print(d)          # {'name': 'Alice', 'age': 30, 'city': '
    Pune'}
print(d['name'])  # Alice
```

Dictionaries operations

- empty `d1 = {}`
- two-item `d2 = {'spam': 2, 'eggs': 3}`
- nesting `d3 = {'food': {'ham': 1, 'egg': 2}}`
- indexing `d2['eggs'], d3['food']['ham']`
- membership `'eggs' in d2` (Python 3 removed `has_key()`), methods `d2.keys(), d2.values()`
- length `len(d1)`
- add/change `d2[key] = new`
- deleting `del d2[key]`

Default Dict

- Is like a regular dictionary, except that when you try to look up a key it doesn't contain, it first adds a value for it using a zero-argument function you provided when you created it.
- Can also be useful with list or dict or even your own functions
- When 2 was not present above, added it with value as empty list, to which 1 was added

```
word_counts = defaultdict(int) # assigns 0 for the newly
    added key
for word in document:
  word_counts[word] += 1

ddlist = defaultdict(list) # assigns empty list for the
    newly added key
ddlist[2].append(1)
```

Counter

- A Counter turns a sequence of values into a defaultdict(int)-like object mapping keys to counts.
- This gives us a very simple way to solve our word_counts problem

```
from collections import Counter
c = Counter([0, 1, 2, 0]) # c is (basically) { 0 :  2, 1 :
    1, 2 : 1}

word_counts = Counter(document)
```

tuples

- They are like lists but immutable.
- When you want to make sure the content won't change.
- Advantage: immutability makes them hashable, so they can be used as dictionary keys or set members (lists cannot); it also signals to readers "this data is fixed"

```
t = (1, 2, 3)
print(t[0])      # 1
# t[0] = 99      # TypeError: tuples are immutable
print(t + (4,))  # (1, 2, 3, 4)
```

Sets

- Unordered collection of unique, hashable objects
- No duplicates: adding an existing value has no effect
- Supports mathematical set operations: union, intersection, difference

```
s = {1, 2, 3, 2, 1}
print(s)                # {1, 2, 3}
s.add(4)
print(3 in s)           # True
print({1,2,3} | {3,4})  # {1, 2, 3, 4}  union
print({1,2,3} & {2,3})  # {2, 3}        intersection
```

Zip and Argument Unpacking

- zip transforms multiple lists into a single list of tuples
- If the lists are different lengths, zip stops as soon as the first list ends.
- You can also unzip: the asterisk performs argument unpacking

```
list1 = ['a', 'b', 'c']
list2 = [1, 2, 3]
zip(list1, list2) # an iterator in Python 3
list(zip(list1, list2)) # [('a', 1), ('b', 2), ('c', 3)]

pairs = [('a', 1), ('b', 2), ('c', 3)]
letters, numbers = zip(*pairs)
```

Files

- input `input = open('data', 'r')`
- read all `S = input.read()`
- read N bytes `S = input.read(N)`
- read next `S = input.readline()`
- read in lists `L = input.readlines()`
- output `output = open('/tmp/spam', 'w')`
- write `output.write(S)`
- write `strings output.writelines(L)`
- close `output.close()`

Unsupported Types

- no char or single byte, use strings of length one or integers
- no pointer
- int vs. short vs. long, only one integer type, arbitrary precision (not tied to C's `long`)
- float vs. double, only one floating point type (C double)

Comparisons vs. Equality

- `L1 = [1, ('a', 3)]`
- `L2 = [1, ('a', 3)]`
- `L1 == L2` is 1
- The `==` operator tests value equivalence
- `L1 is L2` is 0
- The `is` operator tests object identity

Intuition

== asks "do these look the same?", `is` asks "are these literally the same object in memory?". Two separate lists can have identical contents (== true) while being two distinct objects (`is` false) – like two photocopies of the same document being equal in content but not the same physical page.

Control Flow, Functions & Classes

Control Flow, Functions & Classes

if, elif, else

```
if not done and (x > 1):
  doit()
elif done and (x <= 1):
  dothis()
else:
  dothat()
```

while, break

```
while 1:
  line = ReadLine()
  if len(line) == 0:
    break
```

for

Iterates over strings, lists, and ranges (with optional start/stop/step) the same way:

```
# string
for letter in `hello world':
  print(letter)

# list
for item in [12, 'test', 0.1+1.2J]:
  print(item)

# range with bounds and step
for i in range(2,10,2):
  print(i)
```

for vs. the C loop

Python's `for i in range(2,10,2):` is equivalent to:

```
for (i = 2; i < 10; i+=2){
  printf("%d\n",i);
}
```

pass

Temporary filler, the stub. Functions, for loop, wherever there is ":", then on the indented next line *pass* can be put.

```
pass
```

Errors and exceptions

- NameError attempt to access an undeclared variable
- ZeroDivisionError division by any numeric zero
- SyntaxError Python interpreter syntax error
- IndexError request for an out-of-range index for sequence
- KeyError request for a non-existent dictionary key
- IOError input/output error
- AttributeError attempt to access an unknown object attribute

```
try:
  f = open('blah')
except IOError:
  print('could not open file')
```

Functions

- Functions can return any type of object.
- When nothing is return the None object is returned by default.
- Multiple values can be returned.
- Anonymous functions "lambda".
- Parameters can have default arguments.
- Variable-length arguments are supported.

```
def test(a,b=2,d=func):
    return d(a,b)

test(3)
test(b=4,a=3)
test(1,2,lambda x,y: x*y)
test(1,2,g)
```

Modules, namespaces and packages

- A file is a module, e.g. 'myio.py', with a function 'load'; code in it lives in the 'myio' namespace
- To use that function from another file, import the whole module or selectively import just the name
- Packages are bundle of modules.

```
import myio
myio.load()

from myio import load
load()
```

Class

```
class Cone(SomeParentClass):
  def __init__(self,d0,de,L):
    self.a0 = d0/2
    self.ae = de/2
    self.L = L
  def __del__(self):
    pass
  def radius(self,z):
    return self.ae + (self.a0-self.ae)*z/self.L
  def radiusp(self,z):
    return (self.a0-self.ae)/self.L

c = Cone(0.1,0.2,1.5)
c.radius(0.5)
```

Advanced Constructs & Libraries

Advanced Constructs & Libraries

List Comprehensions

Transform a list into another list, and similarly for dict/set comprehensions

```
even_numbers = [x for x in range(5) if x % 2 == 0]   # [0,
      2, 4]
squares = [x*x for x in range(5)] # [0, 1, 4, 9, 16]
even_squares = [x * x for x in even_numbers] # [0, 4, 16]

square_dict = {x:x*x for x in range(5) } # { 0:0, 1:1,
     2:4, 3:9, 4:16 }
square_set = { x*x for x in [1,-1] }  # { 1 }
```

List Comprehensions (more patterns)

Need not use all the values, and can use multiple for loops

```
zeros = [0 for _ in even_numbers] # just to have same size

pairs = [(x,y)
    for x in range(10)
      for y in range(10)] # 100 pairs of [(0,0), (0,1)...]
```

Not so basic: Generators

- List can grow big, very big, which can be a huge source of inefficiency, especially if you need only few values
- Generator iterate over and produce as and when needed
- Create using function and keyword *yield*; consumed one at a time via a for loop until none are left
- Python has ready function like that called *range()*

Intuition

A list computes and stores every value up front, like printing an entire phone book before anyone reads a single number. A generator computes one value only when asked, like reading numbers aloud on demand – `yield` pauses the function exactly where it is, hands back one value, and resumes from that same point next time it's asked for another.

Not so basic: Generators

```
def lazy_range(n):
    """a lazy version of range"""
    i = 0
    while i < n:
        yield i
        i += 1

for i in lazy_range(10):
  do_something_with(i)
```

Not so basic: Randomness

- Many times you need to use random numbers
- Actually produces pseudo-random (that is, deterministic) numbers based on an internal state that you can set with random.seed

```
import  random
four_uniform_randoms = [random.random() for _ in range(4)]
# [0.8444218515250481,   # random.random() produces
     numbers
# 0.7579544029403025, # uniformly between 0 and 1
# 0.420571580830845, #  it's the random function we'll use
# 0.25891675029296335]   # most often

random.seed(10) # set the seed to 10
print(random.random()) # 0.57140259469
random.seed(10) # reset the seed to 10
print(random.random()) # 0.57140259469 again
```

Standard library core modules

- **os** file and process operations.
- **time** dates and times related functions.
- **string** commonly used string operations.
- **re** regular expressions.
- **copy** allow to copy object.

Applications

Applications

Where is Python Used?

- Data Science & Machine Learning: analysis, modeling, deep learning
- Web Development: server-side apps and APIs
- Automation & Scripting: everyday task automation, DevOps tooling
- Scientific & Engineering Computing: simulation, numerical analysis
- Testing: automated test scripts, QA tooling
- Desktop GUIs and simple games
- Glue language: stitching together components written in C/C++/Java

Popular Libraries by Domain

- Data Science/ML: `NumPy, Pandas, scikit-learn, PyTorch`
- Web: `Django, Flask, FastAPI`
- Automation/Testing: `Selenium, pytest`
- Scientific Computing: `NumPy, SciPy, SymPy`
- Visualization: `Matplotlib, Seaborn, Plotly`
- GUI: `Tkinter, PyQt`
- Notebooks: `Jupyter Notebook`

Quick Check: Applications

Think About It

Python shows up in data science, web backends, automation, and glue code alike. What lets one language stretch across such different jobs, and what's the cost of that breadth?

Quick Check: Applications

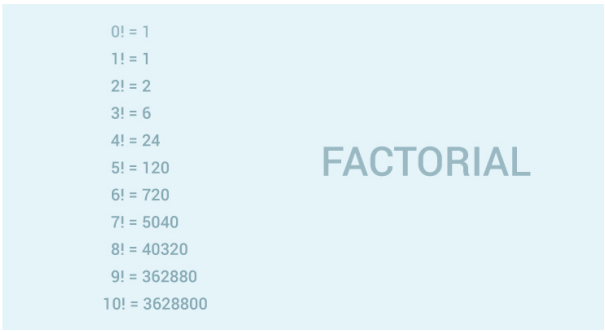
Answer

A simple, readable syntax plus a huge standard library and an enormous third-party ecosystem (via pip) mean there's rarely a need to switch languages just because the task changed. The cost: Python usually isn't the fastest or most memory-efficient choice for any one of those jobs individually, it wins on developer time and flexibility, not raw performance.

Practice

Practice

Quiz
Generate a series of factorial numbers up to n



Solution
Without recursion

```
def myfactorial(n):
  factorial = 1
  # check if the number is negative, positive or zero
  if n < 0:
    print("Sorry, factorial does not exist for negative
      numbers")
  elif n == 0:
    return 1
  else:
    for i in range(1,n + 1):
            factorial = factorial*i
  return factorial

for n in range(1, 8):
  print(n, "! =", myfactorial(n))
```

Solution
With recursion

```
def recur_factorial(n):
   if n == 1:
       return n
   else:
       return n*recur_factorial(n-1)

for n in range(1, 8):
  print(n, "! =", recur_factorial(n))
```

Assignment
Guess the Number in less than 6 trials.
Use `number = random.randint(1, 20)`

```
Hello! What is your name?
- Albert
Well, Albert, I am thinking of a number between 1 and 20.
Take a guess.
- 10
Your guess is too high.
Take a guess.
- 2
Your guess is too low.
Take a guess.
- 4
Good job, Albert! You guessed my number in 3 guesses!
```

Assignment
Solution:

```
# This is a guess the number game.
import random

guessesTaken = 0

print('Hello! What is your name?')
myName = input()

number = random.randint(1, 20)
print('Well, ' + myName + ', I am thinking of a number between 1 and 20.')

while guessesTaken < 6:
    print('Take a guess.') # There are four spaces in front of print.
    guess = input()
    guess = int(guess)

    guessesTaken = guessesTaken + 1

    if guess < number:
        print('Your guess is too low.') # There are eight spaces in front of print.

    if guess > number:
        print('Your guess is too high.')

    if guess == number:
        break

if guess == number:
    guessesTaken = str(guessesTaken)
    print('Good job, ' + myName + '! You guessed my number in ' + guessesTaken + ' guesses!')

if guess != number:
    number = str(number)
    print('Nope. The number I was thinking of was ' + number)
```

The Zen of Python
There should be one - and preferably only one - obvious way to do it.
Pythonic way of doing things.